EE2016 Microprocessor Theory & Lab
Fall, 2025

Week5: Interfacing Peripherals: Interrupts & Comparison with Polling

Dr. R. Manivasakan. OWSM Lab, IIT, Madras

Organization

- Interfacing CPU with I/O or memory or any peripherals
  - Memory mapped
  - Port mapped
- Memory mapped I/O
- Memory interface
- Peripheral interface
- KB Interfacing
- Polling
- Interrupt

Interfacing Peripherals with CPU

- Recap
  - Till now
    - From FF to memory block
      - Memory block representation with data, address and control buses
      - Memory along with ALU
    - Execution Unit
      - Internal registers, status registers, multiplier, shifter etc, along with ALU formed EU. Each sub-engines studied in detail
      - Studied EU along with the first instruction MOV, then LDI
      - Studied many example assembly programs
    - Control Unit
      - PC, Stack, SP, Instruction Decoder, hardwired control unit implementation
      - Studied in detail, each sub-engines
  - Together EU with CU formed microprocessor
    - Studied interfacing peripherals through MMIO (Exp 8)
      - Step 1 Addressing
      - Step 2 Handshake: Polling in KB, as example
- What next?
  - Interfacing peripherals, 2nd step interrupt (KB example) & DMA
- Interrupt
- DMA

Introducing Interrupt

- For polling based key-press detection mechanism, the processor needs to continuously poll every msec to detect any key press
  - This keeps the processor busy all the time – even when the keys are not pressed at all for minutes to hours
- Alternative
  - Processor is **Interrupted** when key is pressed (when any of the Ammeters detect current flowing)
  - The keyboard controller nudges the processor to indicate that a key has been pressed – KB controller is proactive here
  - Once the processor receives this interrupt, it then initiates a scan of all the registers in the keyboard PIC.

What is this Processor Interrupt?

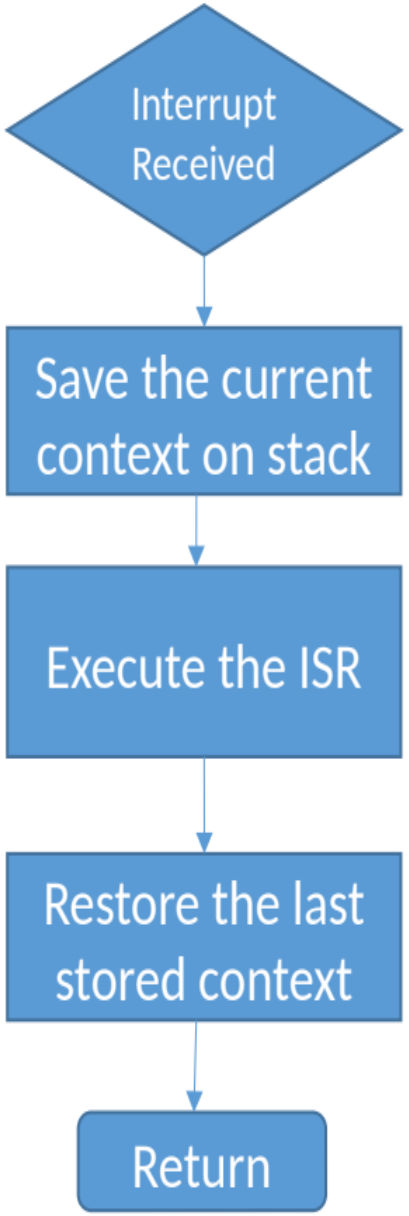
- Processor Execution Unit or Control Unit discussed till now, had till nothing called Interrupt
  - Interrupt will require change in Control Unit (CU)
- An external signal (from PIC) interrupts the Instruction flow-control unit
  - The current Instruction in execution is completed
  - But, then the processor jumps (branch) over to start executing instructions from an "Interrupt" subroutine (instead of currently executing program)
  - Once the task in the Interrupt subroutine is completed, the control unit returns to continue the Interrupted program
    - But this will require the point of interruption to be saved during interrupt and restored during return (in some predefined memory location called STACK)
      - Both the Instruction Pointer as well as all the registers that the subroutine may unwittingly alter, flags are collectively referred to as context of the program

Interrupt: Advantages

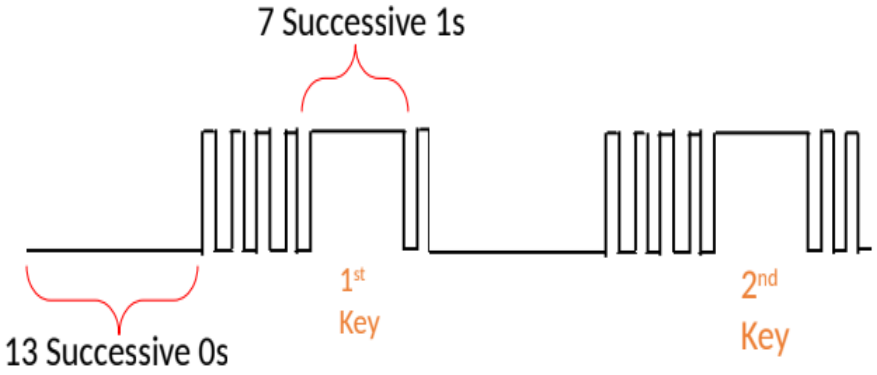
- Advantages of interrupt
  - CPU time is saved as the peripheral is proactive in intimating the CPU that there is an interrupting event
  - CPU energy is saved, since its energy (power) could be spent constructively in executing other programs waiting to be executed
  - CPU throughput (number of instructions executed per sec) is increased.

General Interrupt handling

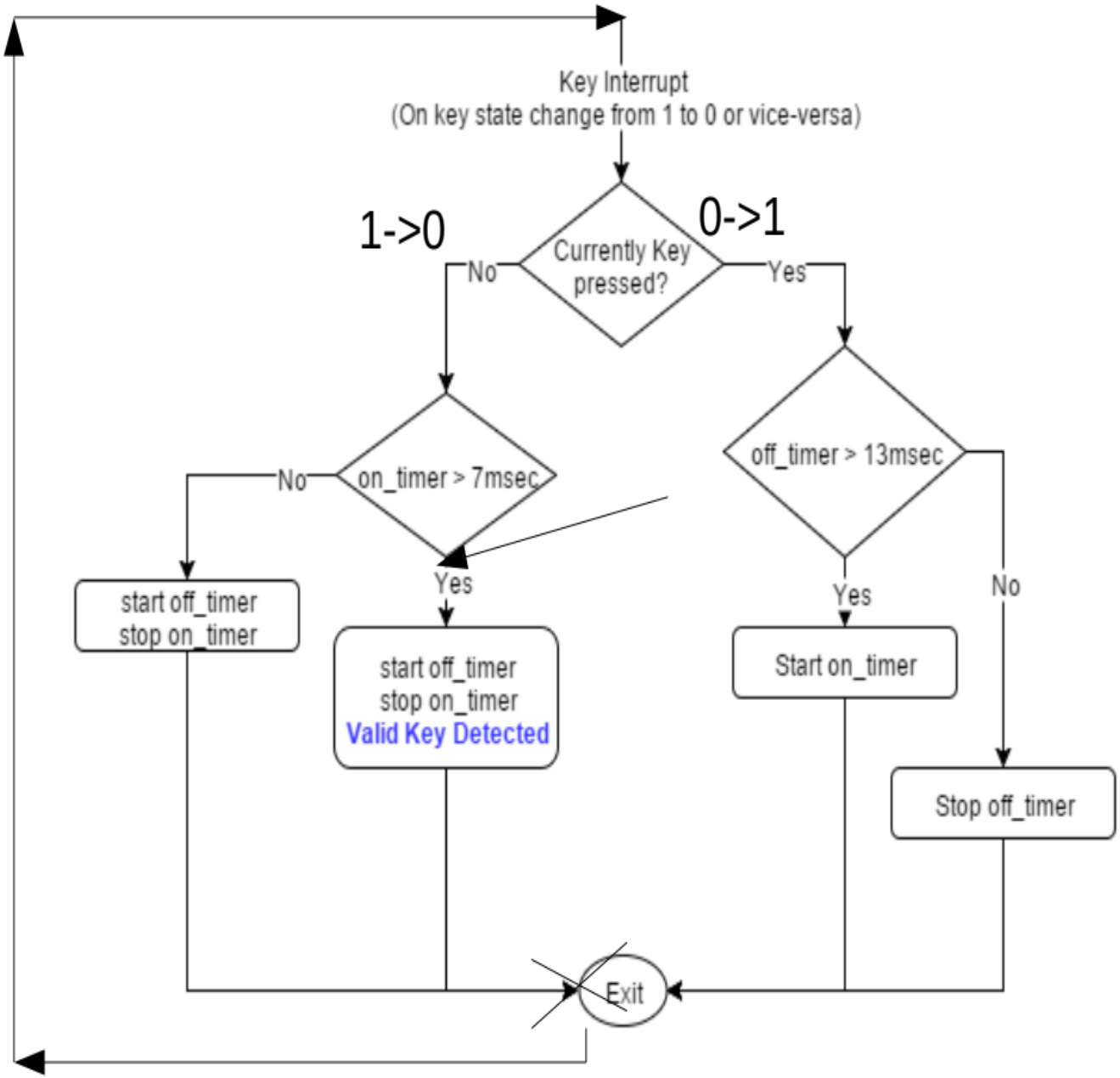
- Interrupt Signal interrupts the processor
  - Can be used by any PIC to draw attention to a task (avoids unnecessary polling)
- Every interrupt has a Interrupt Service Routine (ISR) associated with it.
  - This routine is called whenever an interrupt is received by a processor
- Before executing the ISR, the processor saves the current context: program counter value, Flag, some current register Values
  - This is stored in a stack
- On completion of ISR, the last stored context is restored: populate the PC, flags and saved registers



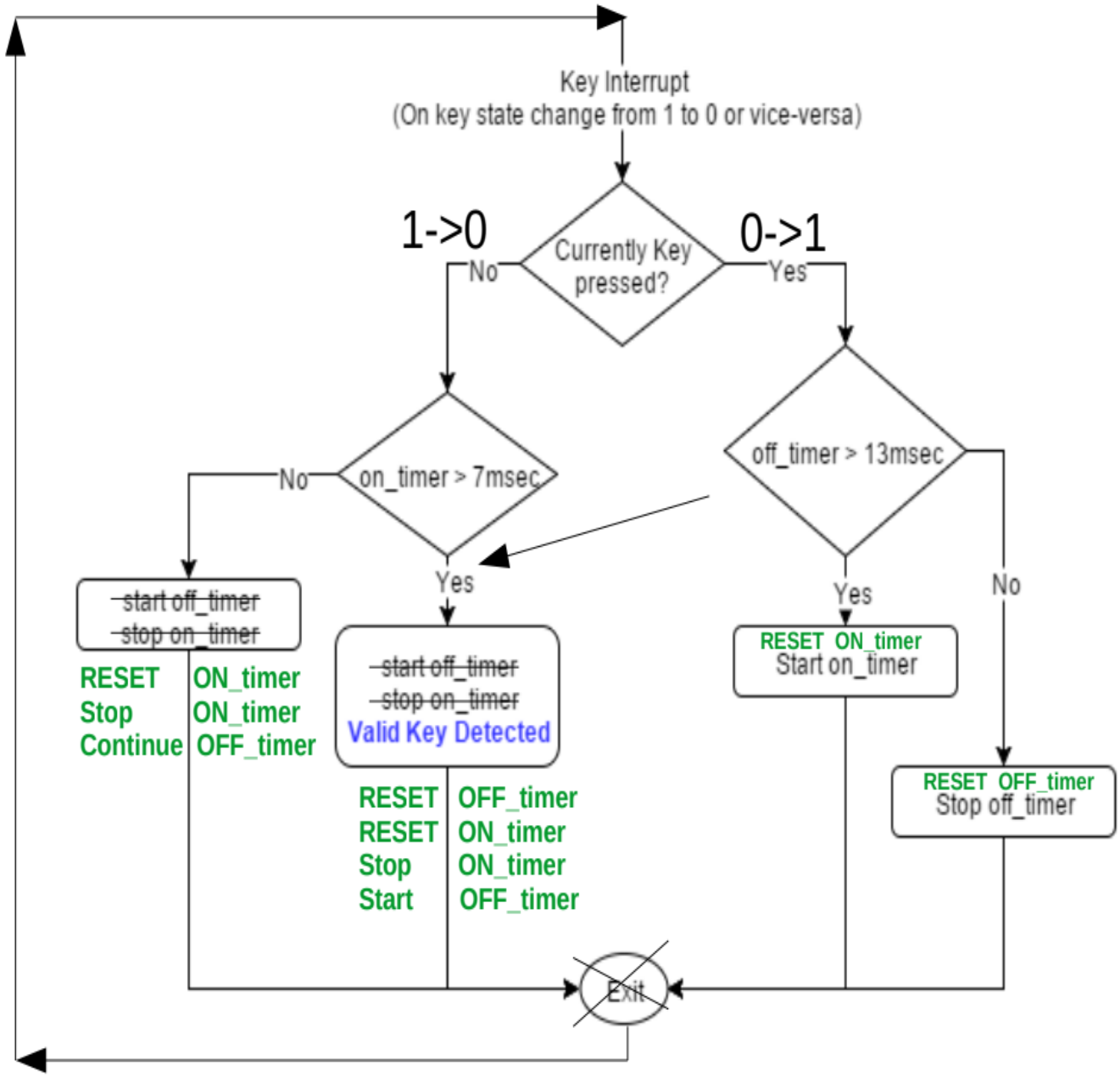
Key Press Detection Using Interrupt



Required Behavior On Consecutive Key Press

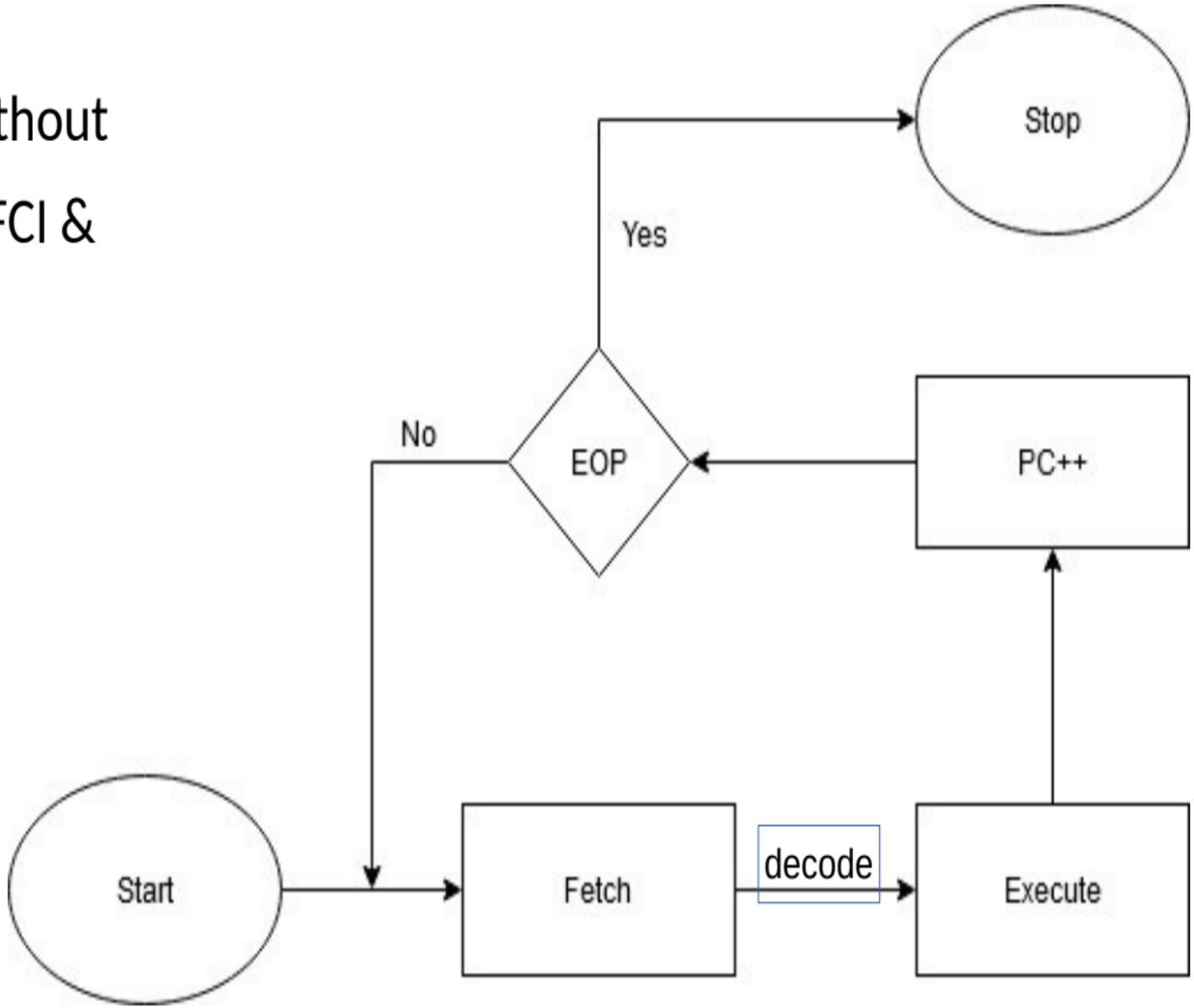


Key Press Detection Using Interrupt



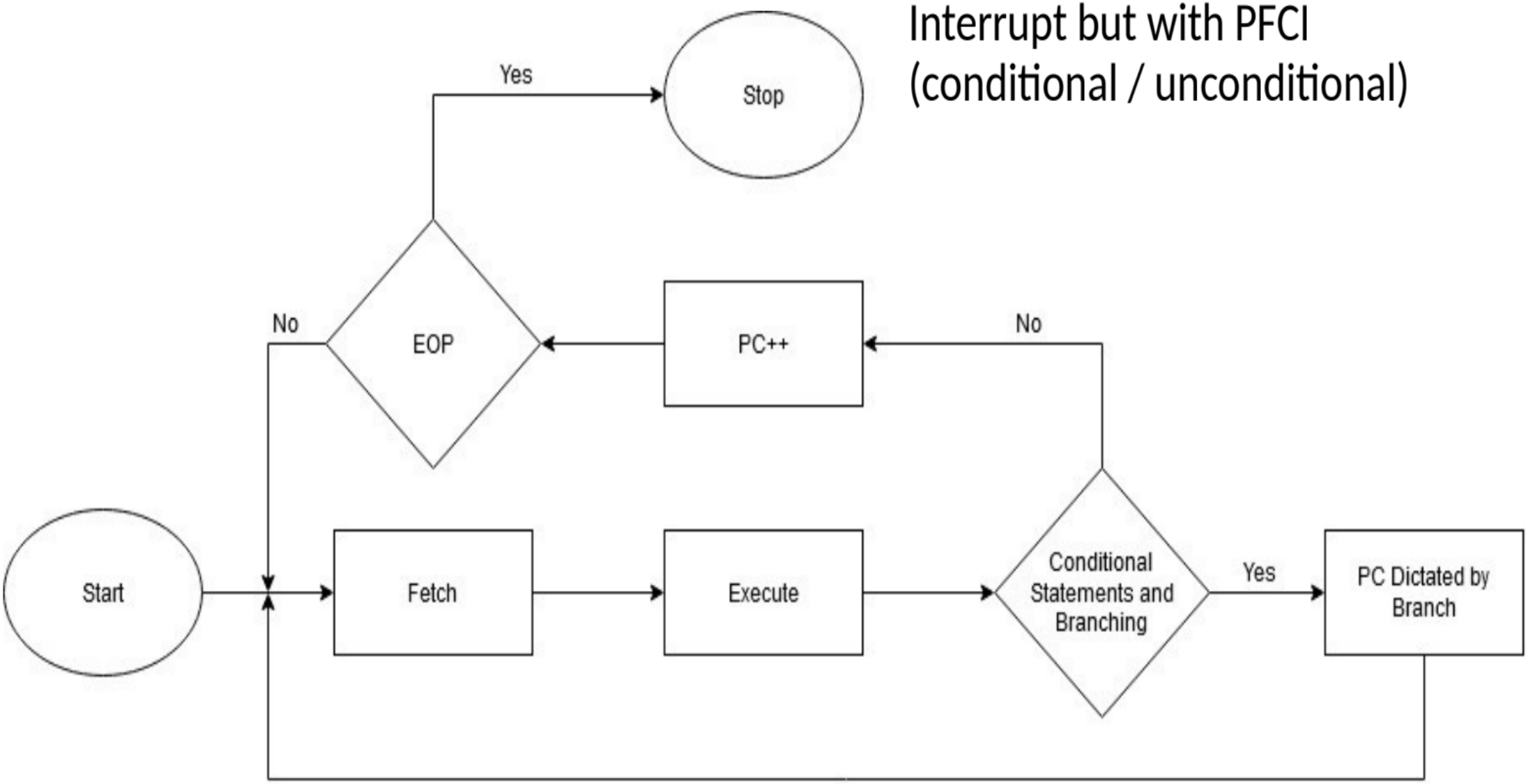
Fetch – Execute Cycle

Fetch – Execute cycle without Interrupt and without PFCI & without ‘Subroutines’



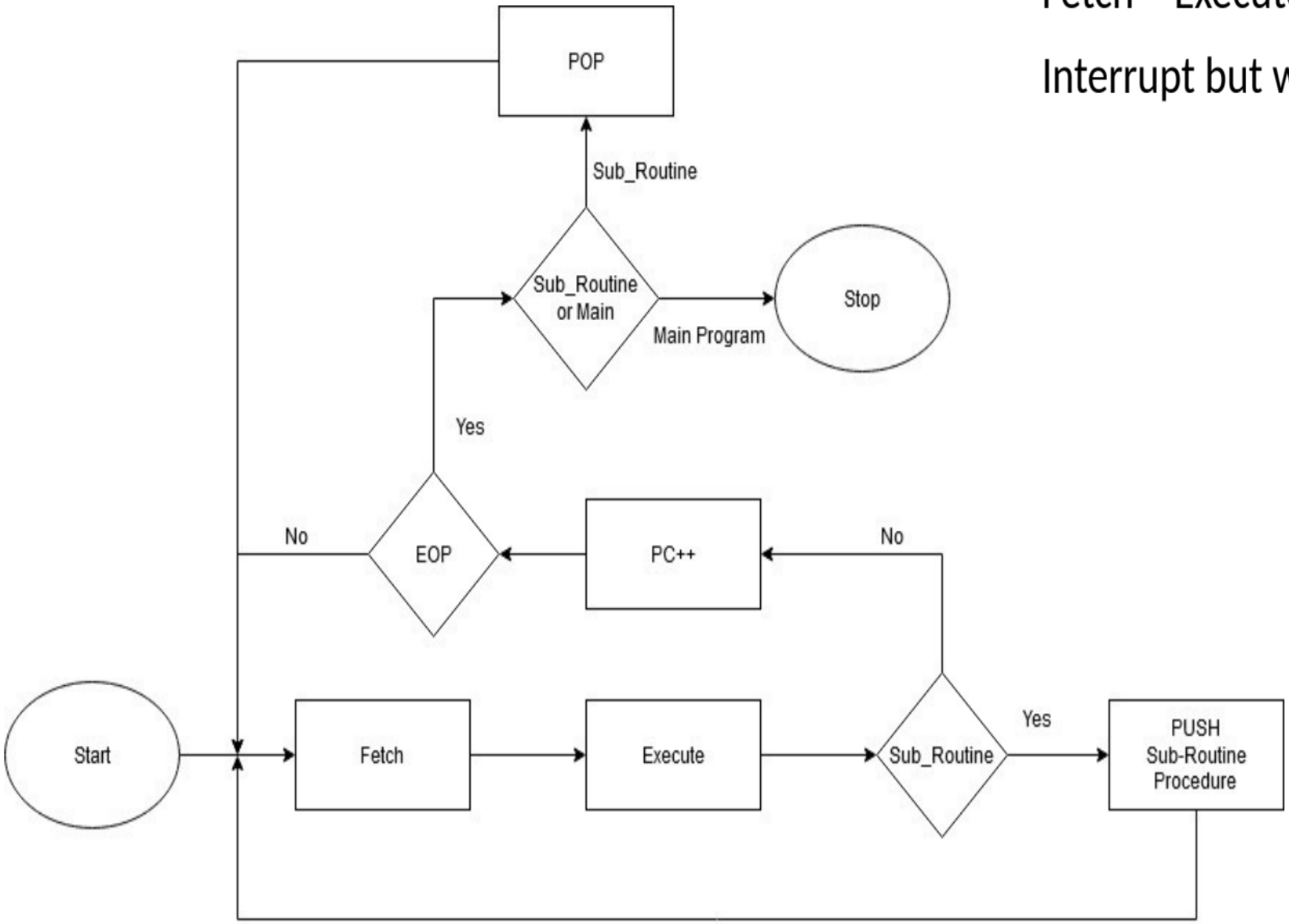
Fetch – Execute Cycle

Fetch – Execute cycle without Interrupt but with PFCI (conditional / unconditional)

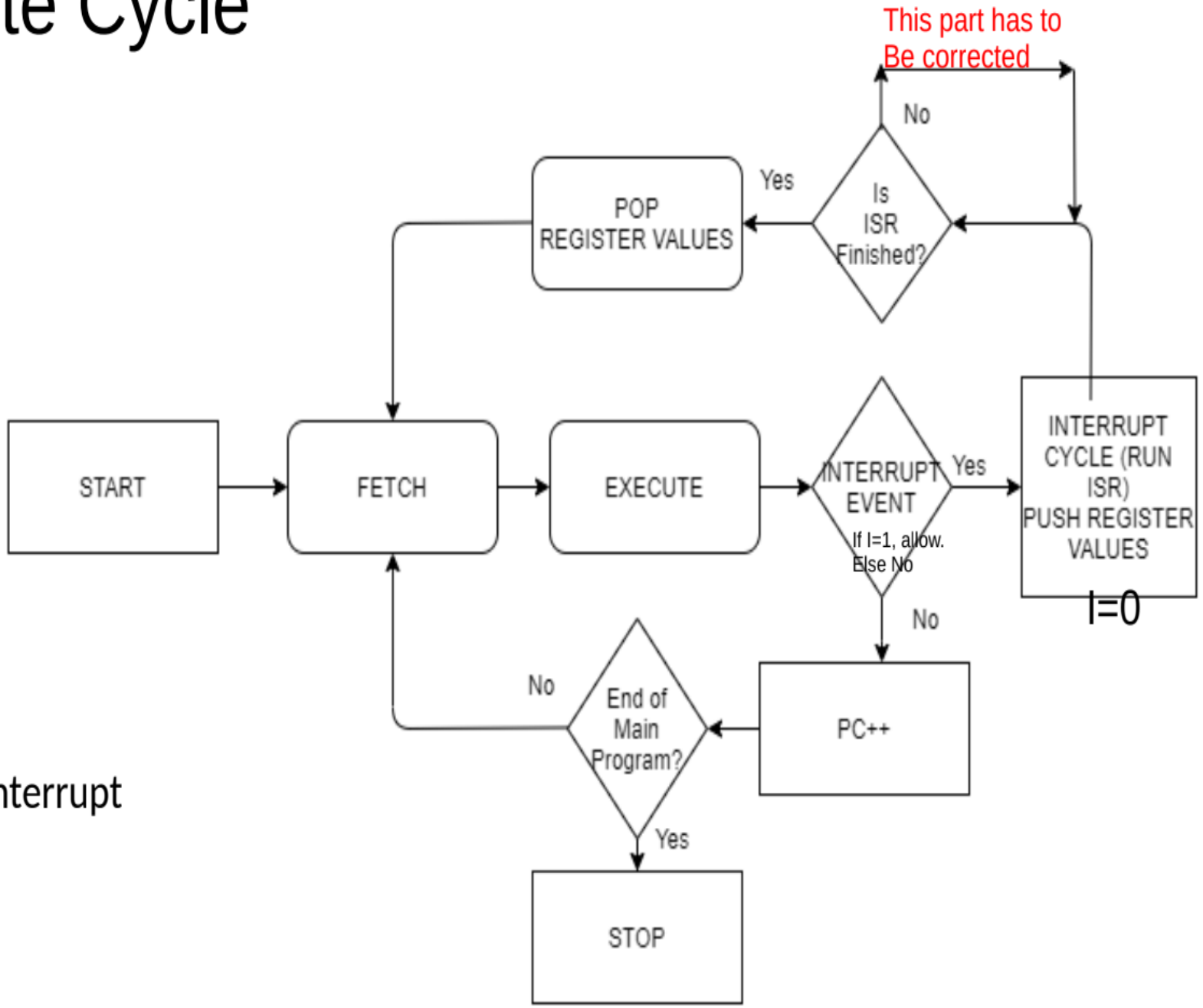


Fetch – Execute Cycle

Fetch – Execute cycle without Interrupt but with subroutine

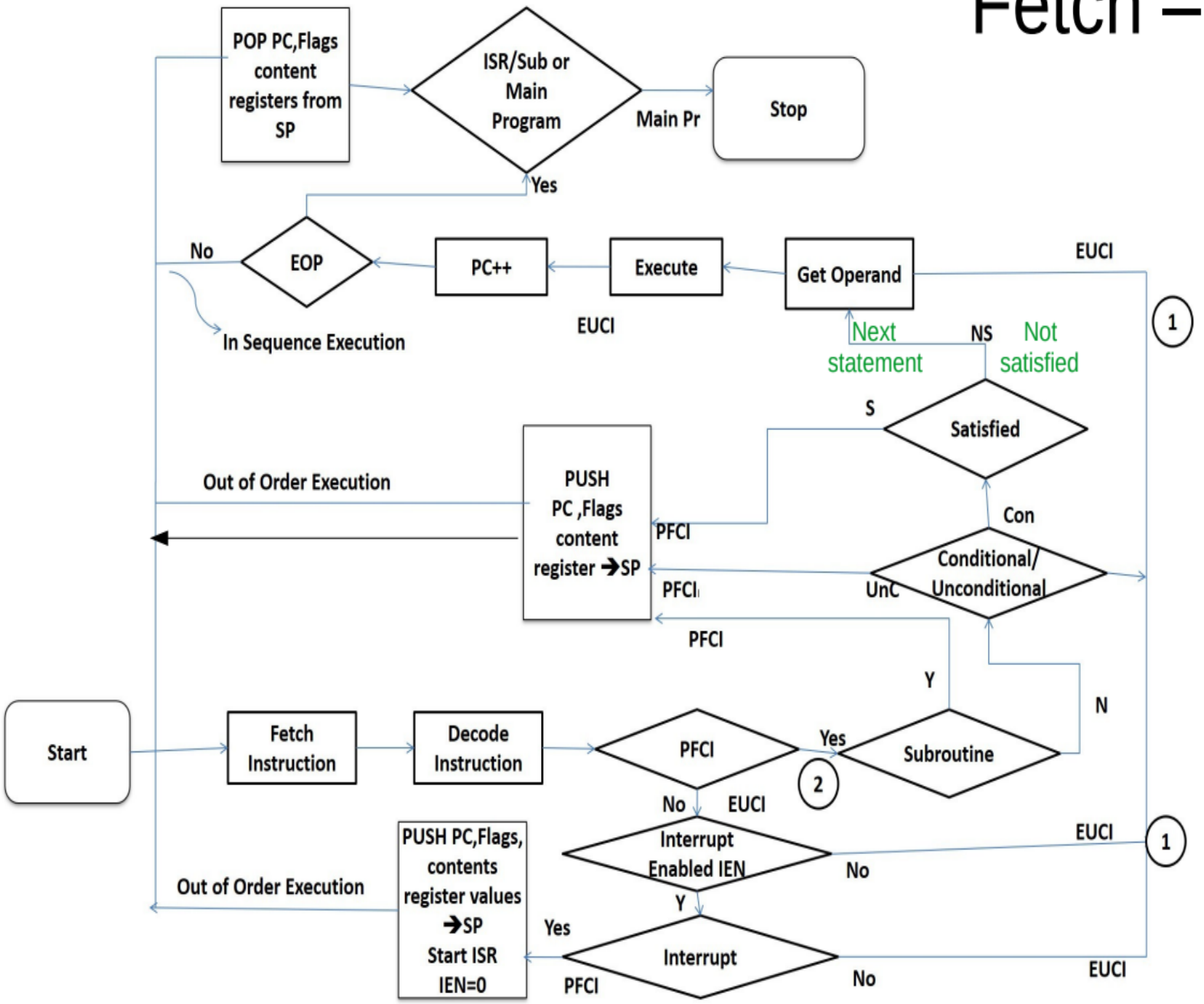


Fetch – Execute Cycle



Fetch – Execute cycle with Interrupt but without subroutine

Fetch – Execute Cycle



- Fetch – Execute cycle with Interrupt & subroutine

STACK POINTER
VERSUS
PROGRAM COUNTER

STACK POINTER	PROGRAM COUNTER
A CPU register whose purpose is to keep track of a call stack	A CPU register that indicates where a computer is in its program sequence
Also called a stack register	Also called instruction pointer, instruction address register, and instruction counter
Holds the address of the last program request in a stack	Holds the address of the next instruction that should be executed
Tracks the operations of the stack	Helps to track the current execution point

Visit www.PEDIAA.com

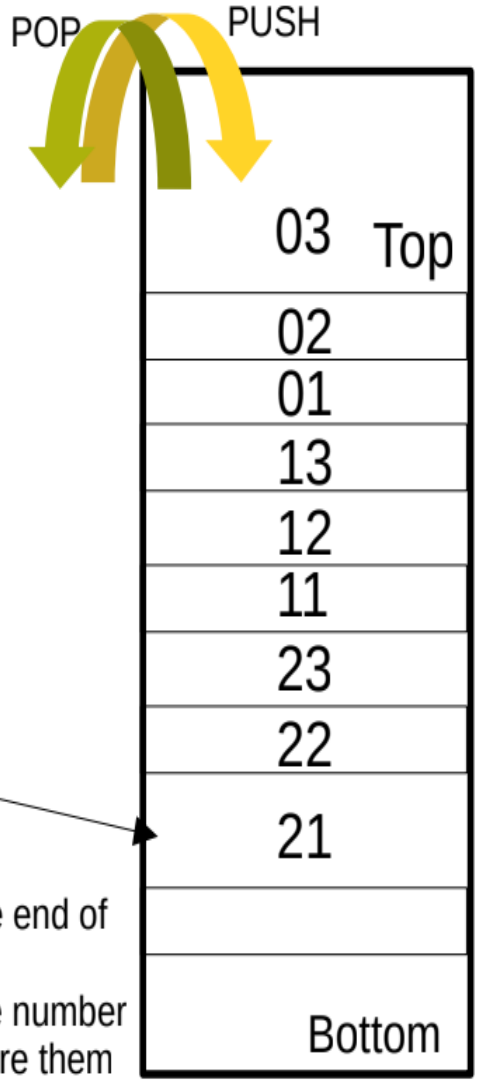
Stack and Stack Pointer

- Stack Memory
  - A piece of memory which could be external SRAM (a part of main memory) or could be internal memory, whose access is restricted / different than that of conventional way of accessing
  - Stack memory is a subset of memory, in which () one end (say, TOP – address of its active part) is fixed, the other (say BOTTOM, address of its active part) could be variable.
- Stack Pointer
  - 2 byte pointer pointing to the variable end of the active part of stack memory
- Stack & Interrupts / subroutines
  - When there is no interrupt / subroutine being served, then stack pointer points to the top end. Interrupts / subroutines can be nested also.
  - As and when the Interrupt arrives, the (a) PC values (b) flag /status register values (c) context registers & local variables are PUSHed into the stack memory from top, in the arrival order and correspondingly the stack pointer also decremented as shown in illustration.
  - When the top most (inner most in the nest) interrupt's (which is currently getting served) ISR is served, then the above three values are POPped up while the stack pointer moves up.
- Assembly syntax
  - PUSH
    - The next instruction to be executed is 'pushed' in
  - POP
    - The stored instructions are "Pop"ped out in the order opposite to the order in which it was stored
    - --> First IN, last OUT queue, or last IN, first OUT
  - Used during the interrupt and subroutine part of the program

INT 0
03 – Context register value
02 – Status register value
01 – PC value

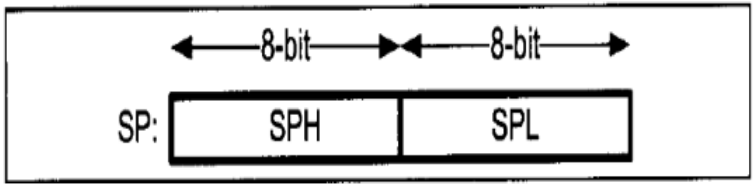
INT 1
13 – Context register value
12 – Status register value
11 – PC value

INT 2
23 – Context register value
22 – Status register value
21 – PC value

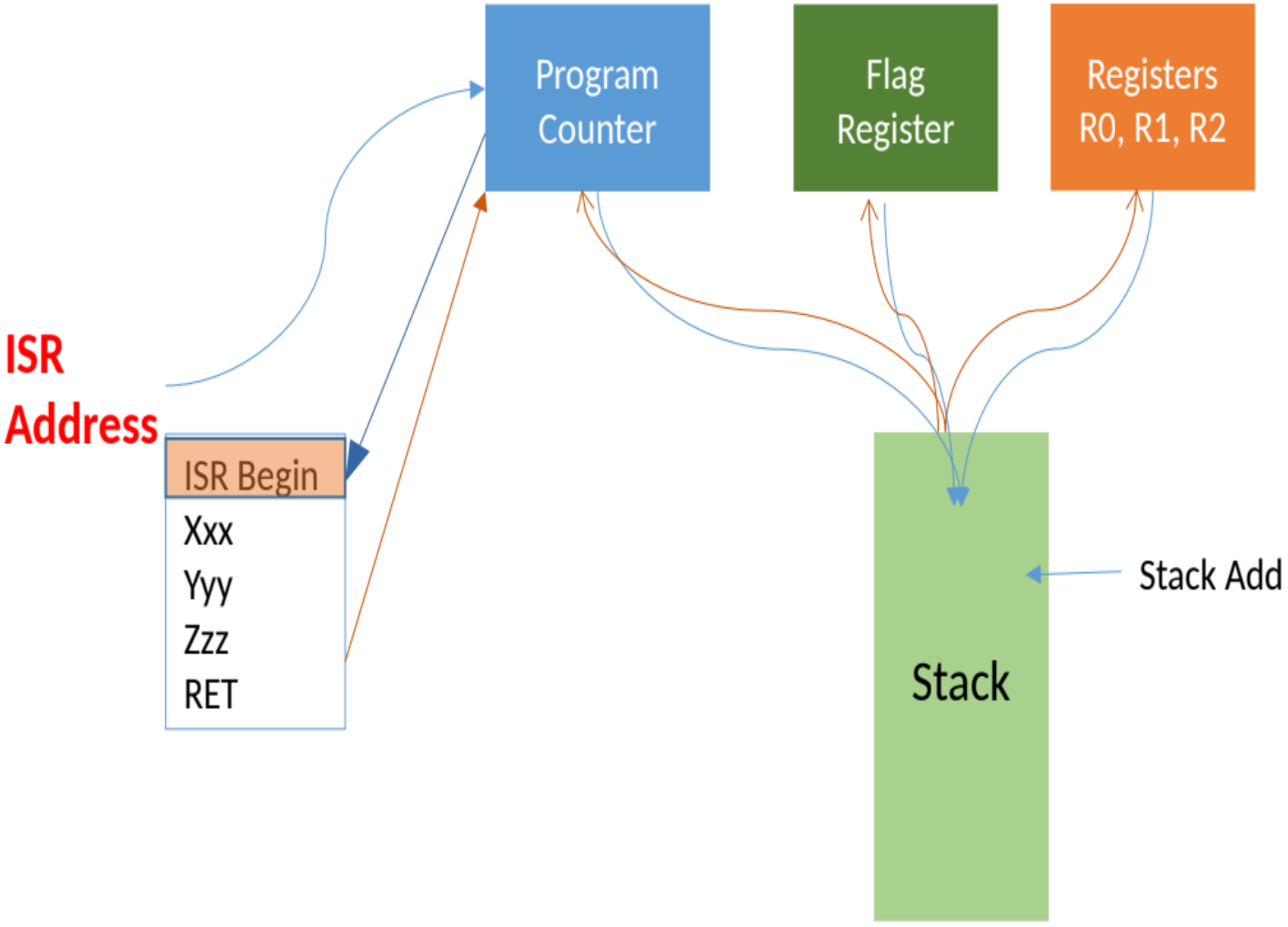


$5C	$3C	OCR0
$5D	$3D	SPL
$5E	$3E	SPH
$5F	$3F	SREG

- Stack pointer ~ address of variable end of stack
- Another byte needed to denote the number of memory locations needed to store them
- (address, 0, 2, 3)



Interrupt Process Flow



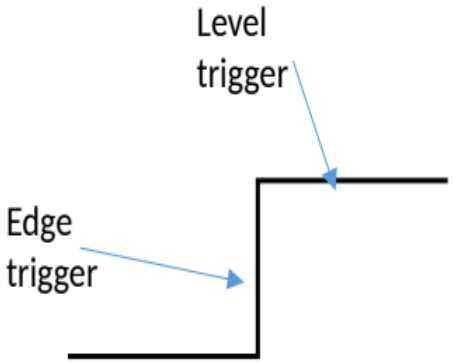
- On Interrupt, the processor PUSHES the current context to Stack

INT

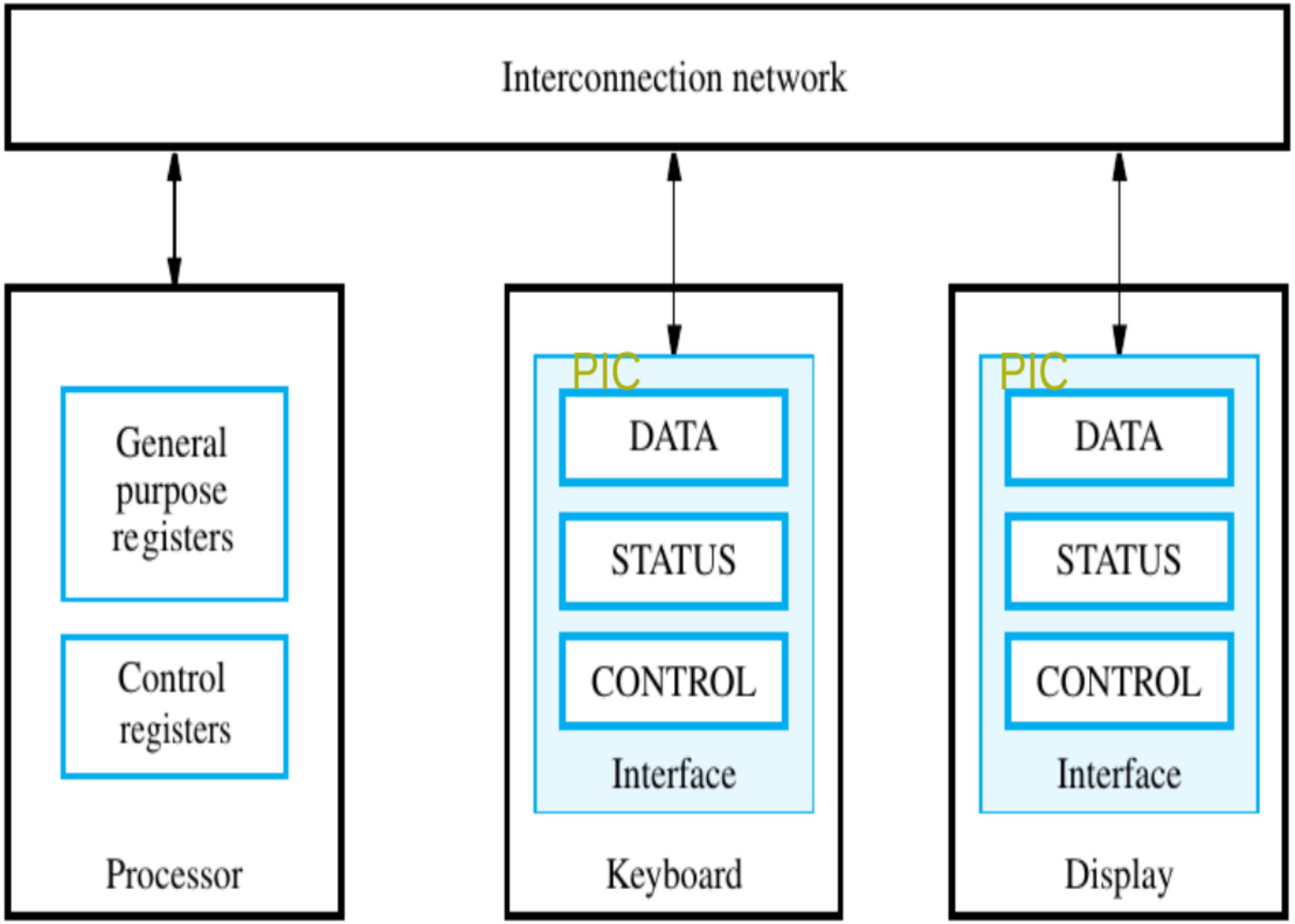
- On Return, the processor POPS the last saved current context out of Stack
- Stack is Last IN First OUT memory
  - Stack Address is incremented on Push and decremented on Pop
- Time taken for interrupt handling: Time for context switch + Time taken to execute the ISR

Nested Interrupts

- A Processor may have multiple Interrupts
  - Associated with multiple ISR tasks, triggered by different external events
  - An Interrupt may interrupt an ISR: Nested Interrupts
    - Ongoing ISR is interrupted with PC and context pushed into stack; interrupting subroutine will be executed and the previous ISR will be continued
    - Stack will grow further to accommodate multiple contexts and shrink when ISR exits
- Different interrupts may have different priorities
  - Only an higher priority interrupt may interrupt an ongoing ISR
    - Equal and lower priority interrupt will be kept pending till earlier ISR exits
- Interrupt may be edge triggered or Level triggered
  - Edge triggering for immediate response



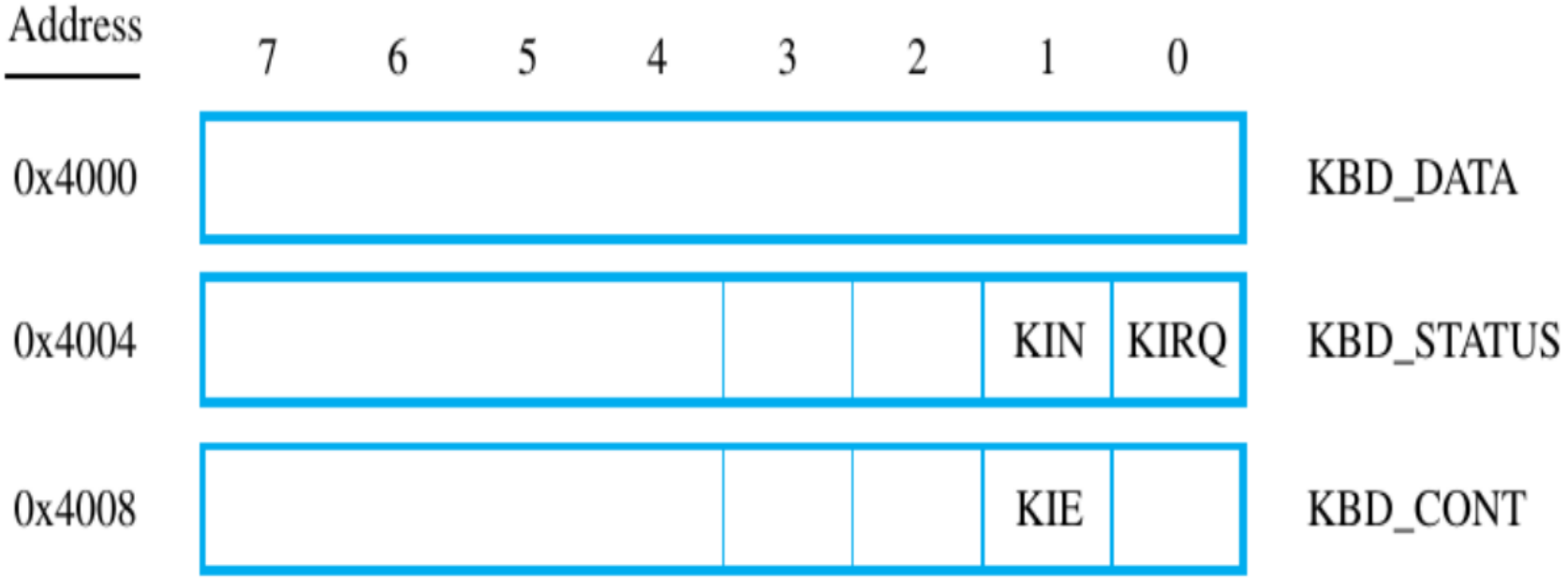
Interconnecting Processor, Keyboard & Display



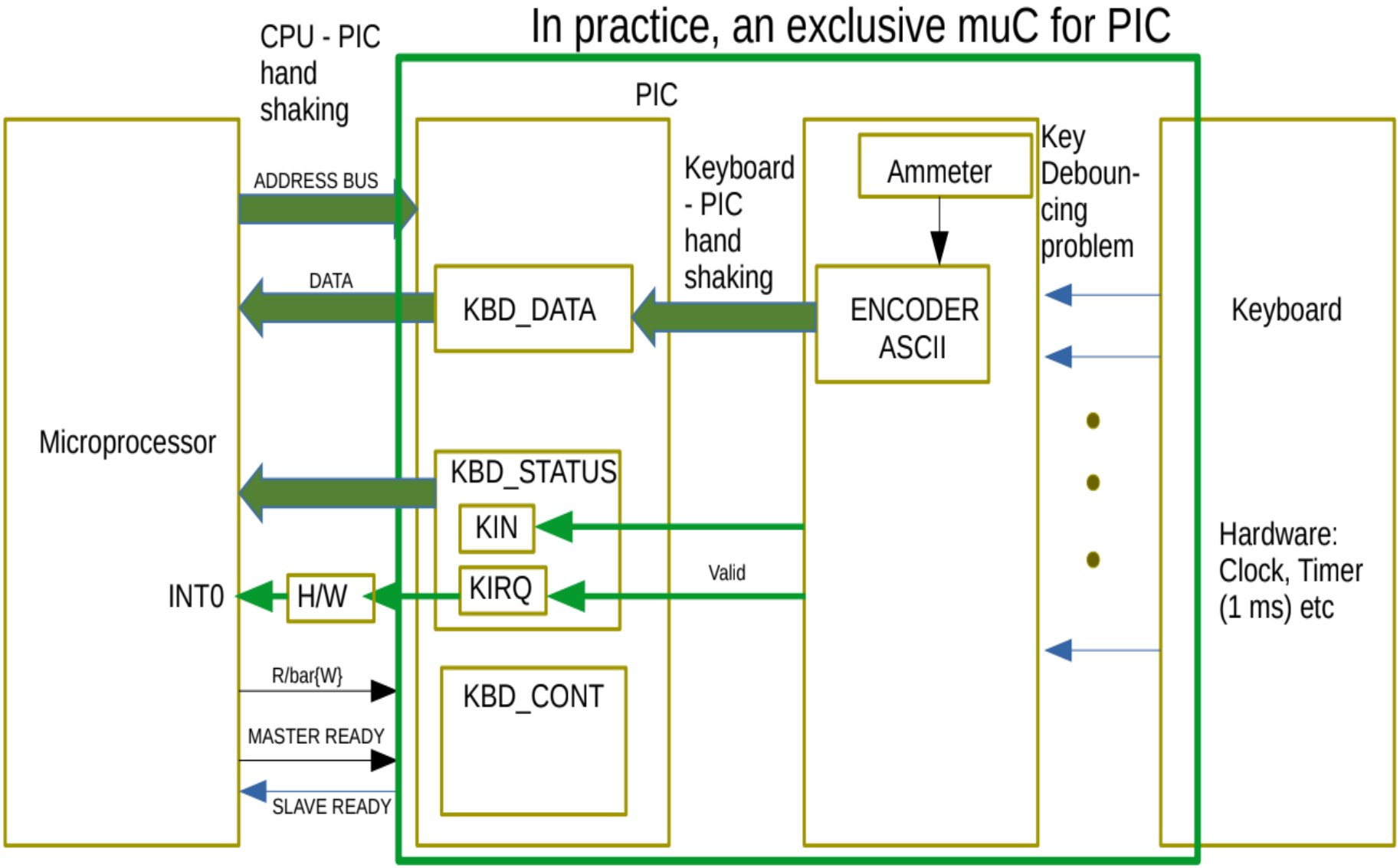
Keyboard PIC Registers

Hamachar, P99

- KBD_DATA
  - Data register holding ASCII code of the key pressed latest, in the keyboard
- KBD_STATUS
  - Status of the keyboard – meaning whether the key pressed has already read by the CPU (& hence the data in the KBD_DATA is invalid)
  - KIN – Keyboard input status bit, set to 1 if ASCII code word corresponding to the key pressed is yet to be read by CPU
  - KIRQ - The KIRQ bit is set to 1 if an interrupt request has been raised, but not yet serviced
- KBD_CONT
  - Keyboard control register – controls or configures the keyboard function
  - KIE – bit number 1 in KBD_CONT
    - Keyboard interrupt enable (KIE)
    - Interface the keyboard to the CPU through interrupt
- Keyboard Interfacing through MMIO



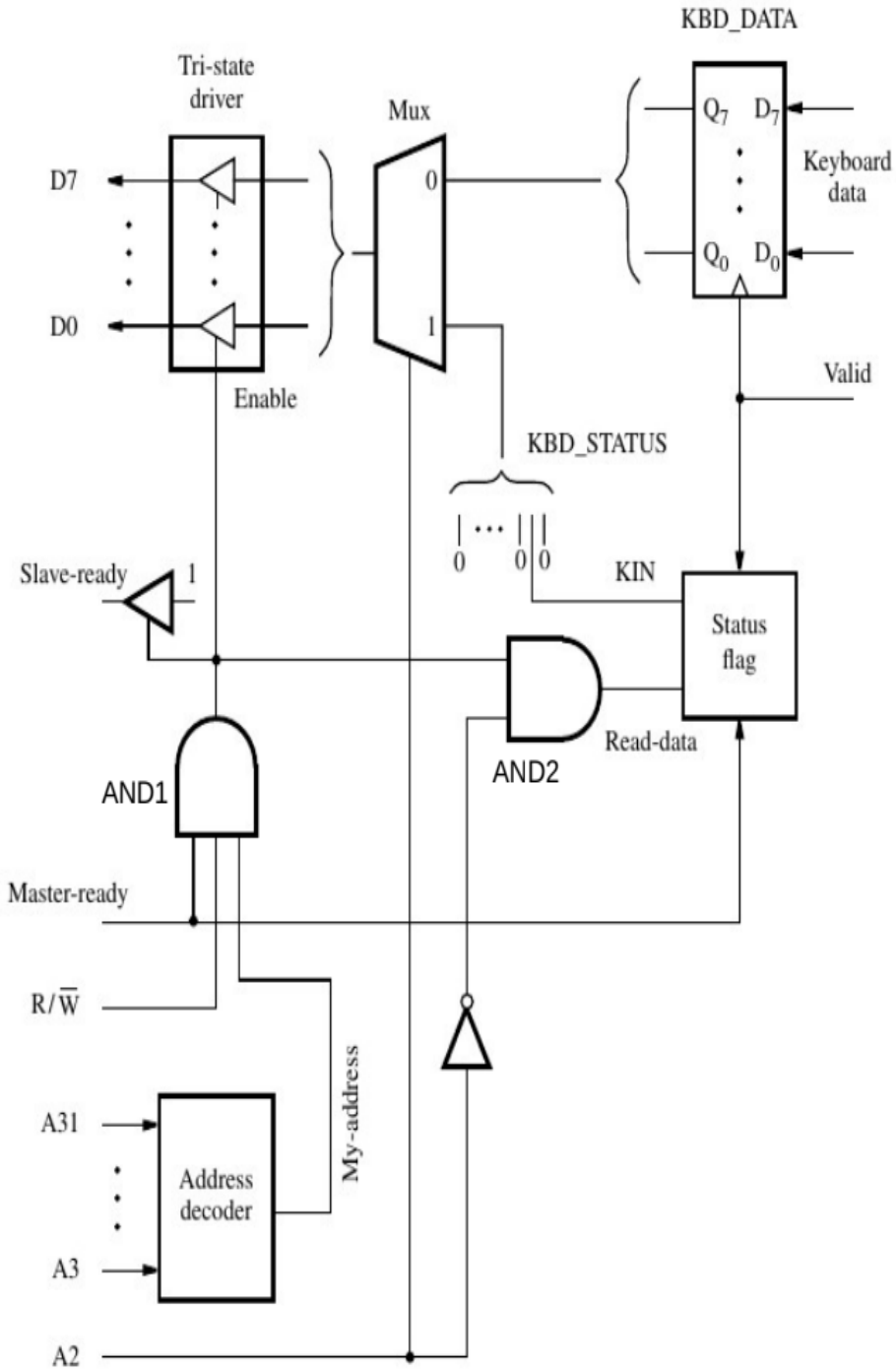
Key Board Interface



Keyboard Interfacing

- Keyboard Interfacing Problem
  - Key De-Bouncing problem
  - KB interfacing problem through PIC
  - Implementation
    - Key debouncing problem – an exclusive microcontroller
    - In general purpose muP based systems, like PC, PIC is handled by CPU (through external interrupt PIN - eg. INT0 as in AVR – or by polling)
- Key Debouncing problem – an exclusive microcontroller
  - Polling (1 ms intervals)
  - Interrupt (continuous – not really?)
- CPU – KB interfacing problem through PIC
  - Polling (KIN bit polling at 1 ms intervals)
  - Interrupt (KIRQ through special hardware to feed input to external interrupt as in, for eg. INT0 in AVR)
  - CPU attends to KIRQ (Keyboard Interrupt Request) to receive the ASCII code and displays in the monitor (which is a separate and independent problem).

Keyboard Interfacing Circuitry



To Conclude

- Polling registers in Peripheral Controller used to detect state of Peripherals
  - Would require frequent polling and waste of CPU cycles
- Interrupt is an alternative means to draw attention to an external event occurring at a peripheral
  - Enables CPU to suspend its current task and handle external event through Interrupt subroutine (ISR) which reads status / data registers of a peripheral controller
  - CPU would resume suspended task after the ISR exits, without any glitch
  - Nested Interrupts and Interrupt Priority used to handle interrupts form multiple external events
- Interrupt Controller chip (like a Peripheral Controller) sometime used to connect multiple interrupts, define priorities for each interrupt and turn on / off any Interrupt